Nexo

Arquitetura do banco de dados

Diagrama de relacionamentos, as treze tabelas com campos, tipos, chaves e restrições, os índices, as bases de conhecimento, os agentes, o WhatsApp, a migração automática da 2.0, a coleção vetorial, o arquivo de contas, as consultas e os caminhos para trocar de banco e escalar.

Versão 2.1.0
Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01	**Visão geral**	**5**
1.1	Três lugares, três papéis	5
1.2	Criação e migração automáticas	5
1.3	Convenções	5
1.4	Tabelas por área	6
1.5	O que mudou na 2.1	6
02	**Diagrama de relacionamentos**	**7**
2.1	Relacionamentos	7
2.2	Diagrama	8
03	**Bases e acervo**	**9**
3.1	nx_knowledge_base	9
3.2	nx_document	9
3.3	Ciclo do status	10
04	**Agentes**	**11**
4.1	nx_bot	11
4.2	Como o agente usa a base	11
05	**Conversas e mensagens**	**12**
5.1	nx_conversation	12
5.2	nx_conversation_document	12
5.3	nx_message	12
5.4	nx_message_source	13
5.5	nx_feedback	13
06	**WhatsApp**	**14**
6.1	nx_whatsapp_channel	14
6.2	nx_whatsapp_contact	15
6.3	nx_whatsapp_message	15
6.4	Como uma mensagem é gravada	16
07	**Integrações**	**17**

Copyright (©) 2026 Erik Pimenta

7.1	nx_api_key	17
7.2	Por que só o hash da chave é guardado	17
7.3	nx_webhook	18
7.4	Como as entregas gravam last_status	18
08	**Banco vetorial**	**20**
8.1	Coleção	20
8.2	O que entra no embedding	20
8.3	Filtros e exclusão	20
8.4	Trocar o modelo de embeddings	21
09	**Contas e arquivos**	**22**
9.1	data/usuarios.json	22
9.2	data/uploads	23
9.3	data/.segredo	23
10	**Consultas e índices**	**24**
10.1	Consultas principais	24
10.2	Lista de índices	25
10.3	Consultas úteis da 2.1	25
10.4	Consultas do painel de qualidade	26
11	**Decisões de arquitetura**	**29**
11.1	Relacional e vetorial separados	29
11.2	Fontes gravadas com cópia do texto	29
11.3	Contas em arquivo	29
11.4	IDs gerados pela aplicação	29
11.5	Escopo por ausência	29
11.6	Chaves de API como hash	29
11.7	Webhooks sem fila persistente	30
11.8	Métricas calculadas na hora	30
11.9	Filtro do agente pelos ids dos documentos da base	30
11.10	Segredos do WhatsApp mascarados na API	30
11.11	Único de sha256 por base	30
12	**Migração automática**	**31**

Copyright (©) 2026 Erik Pimenta

12.1	O que a migração faz	31
12.2	SQLite: os 12 passos oficiais	31
12.3	Outros bancos	31
13	**Fluxos de dados**	**32**
13.1	Envio de um arquivo para uma base	32
13.2	Uma pergunta a um agente	32
13.3	Pergunta direta e busca	32
13.4	Avaliação	32
13.5	Exclusões	33
14	**Outros bancos e escala**	**34**
14.1	Outro banco relacional	34
14.2	Migrando dados do SQLite	34
14.3	Crescimento	34
14.4	Backup	35
14.5	DDL de referência	35

Copyright (©) 2026 Erik Pimenta

01

Visão geral

O Nexo guarda informações em três lugares, cada um com um papel. Este documento descreve a estrutura que os dados devem seguir, para quem administra o banco, migra para outro servidor ou integra o Nexo com outros sistemas.

1.1 Três lugares, três papéis

Camada	Onde fica (padrão)	O que guarda
Banco relacional	data/nexo.db (SQLite) ou o banco de NEXO_DATABASE_URL	Bases de conhecimento, documentos e textos, agentes, conversas, mensagens, fontes, avaliações, chaves de API, webhooks e o WhatsApp (conexões, contatos e mensagens)
Banco vetorial	data/chroma/ (ChromaDB) ou Qdrant (data/qdrant/ ou servidor)	Trechos dos arquivos com os embeddings e os metadados de origem
Arquivos	data/uploads/<id>.<extensão>	Arquivos originais (PDF, DOCX, TXT, MD, CSV, JSON, HTML), gravados com o id interno
Contas	data/usuarios.json	Usuários, perfis, hashes de senha e quando viram o tutorial
Segredo	data/.segredo	Chave das sessões quando NEXO_SECRET está vazio

A pasta `data` (ou a de `NEXO_DATA_DIR`) concentra tudo o que é local. Copiar essa pasta com o servidor parado é um backup completo.

1.2 Criação e migração automáticas

As tabelas são criadas pelo SQLAlchemy na primeira conexão (`Base.metadata.create_all`). Na 2.1, logo depois, `nexo/migrations.py` confere o banco e aplica o que falta para bancos vindos da 2.0: colunas novas, a base padrão "Acervo geral" e a troca do único de `sha256` pelo único por base (capítulo 12). No SQLite, cada conexão liga `PRAGMA foreign_keys=ON`, para que as regras ON DELETE funcionem, e `PRAGMA journal_mode=WAL`, para que leituras não esperem as gravações da fila. Os arquivos `docs/schema-postgresql.sql` e `docs/schema-sqlite.sql` trazem o DDL equivalente.

1.3 Convenções

Convenção	Regra
Prefixo	Todas as tabelas começam com nx_, para conviver com outras aplicações no mesmo banco

Copyright (©) 2026 Erik Pimenta

Convenção	Regra
Chaves de texto	id de bases, documentos, agentes, conversas, mensagens, chaves, webhooks, conexões, contatos e mensagens do WhatsApp é um UUID4 em hexadecimal, varchar(32), gerado pela aplicação
Chaves numéricas	Fontes e avaliações usam integer autoincremento (SERIAL no PostgreSQL)
Datas	timestamp sem fuso, sempre em UTC; a interface converte para o horário local
Usuários	Colunas owner, uploaded_by e created_by guardam o username do arquivo de contas (não há tabela de usuários)
Estados	Documentos em maiúsculas (PENDING, PROCESSING, READY, FAILED); tipos e situações do WhatsApp em minúsculas (pdf, file, note; in, out; sent, delivered, read, failed)
Listas curtas	Os eventos de um webhook ficam numa coluna de texto, separados por vírgula; as fontes de uma resposta do WhatsApp, em JSON

1.4 Tabelas por área

Área	Tabelas
Bases e acervo	nx_knowledge_base **Nova**, nx_document
Agentes	nx_bot **Nova**
Conversas	nx_conversation, nx_conversation_document
Mensagens	nx_message, nx_message_source
Qualidade	nx_feedback
Integrações	nx_api_key, nx_webhook
WhatsApp	nx_whatsapp_channel **Nova**, nx_whatsapp_contact **Nova**, nx_whatsapp_message **Nova**

1.5 O que mudou na 2.1

- Cinco tabelas novas: `nx_knowledge_base`, `nx_bot`, `nx_whatsapp_channel`, `nx_whatsapp_contact` e `nx_whatsapp_message`.
- `nx_document` ganhou `base_id`, `kind`, `extension`, `content` e `updated_at`; o único de `sha256` passou a valer por base (`uk_document_base_sha256`).
- `nx_conversation` ganhou `bot_id`, que liga a conversa a um agente.
- O arquivo de contas ganhou `tutorial_done_at`.
- O banco vetorial não muda: as bases filtram pelos ids dos documentos, então nenhum vetor precisa ser recalculado.

Copyright (©) 2026 Erik Pimenta

02

Diagrama de relacionamentos

O acervo agora é organizado em bases de conhecimento, e cada agente usa uma base. As conversas podem pertencer a um agente, e o WhatsApp forma um ramo próprio: conexão, contatos e mensagens.

2.1 Relacionamentos

Pai	Filho	Relação	Ao excluir o pai
nx_knowledge_base	nx_document	1 para N	CASCADE no banco; a aplicação apaga antes os vetores e os arquivos
nx_knowledge_base	nx_bot	1 para N	Bloqueado: a API devolve 409 BASE_IN_USE enquanto houver agente usando a base
nx_bot	nx_conversation	0..1 para N	CASCADE: as conversas do agente somem com ele
nx_bot	nx_whatsapp_channel	0..1 para N	SET NULL: a conexão continua, sem resposta automática
nx_whatsapp_channel	nx_whatsapp_contact	1 para N	CASCADE
nx_whatsapp_contact	nx_whatsapp_message	1 para N	CASCADE
nx_conversation	nx_conversation_document	1 para N	CASCADE
nx_document	nx_conversation_document	1 para N	CASCADE: o documento sai do escopo de todas as conversas
nx_conversation	nx_message	1 para N	CASCADE
nx_message	nx_message_source	1 para N	CASCADE
nx_document	nx_message_source	0..1 para N	SET NULL: o título e o trecho continuam no histórico
nx_message	nx_feedback	1 para N	CASCADE (uma avaliação por usuário)
conta (usuarios.json)	nx_api_key, nx_webhook, criadores	1 para N	Sem chave estrangeira; a aplicação apaga chaves e conversas junto com a conta

Copyright (©) 2026 Erik Pimenta

2.2 Diagrama

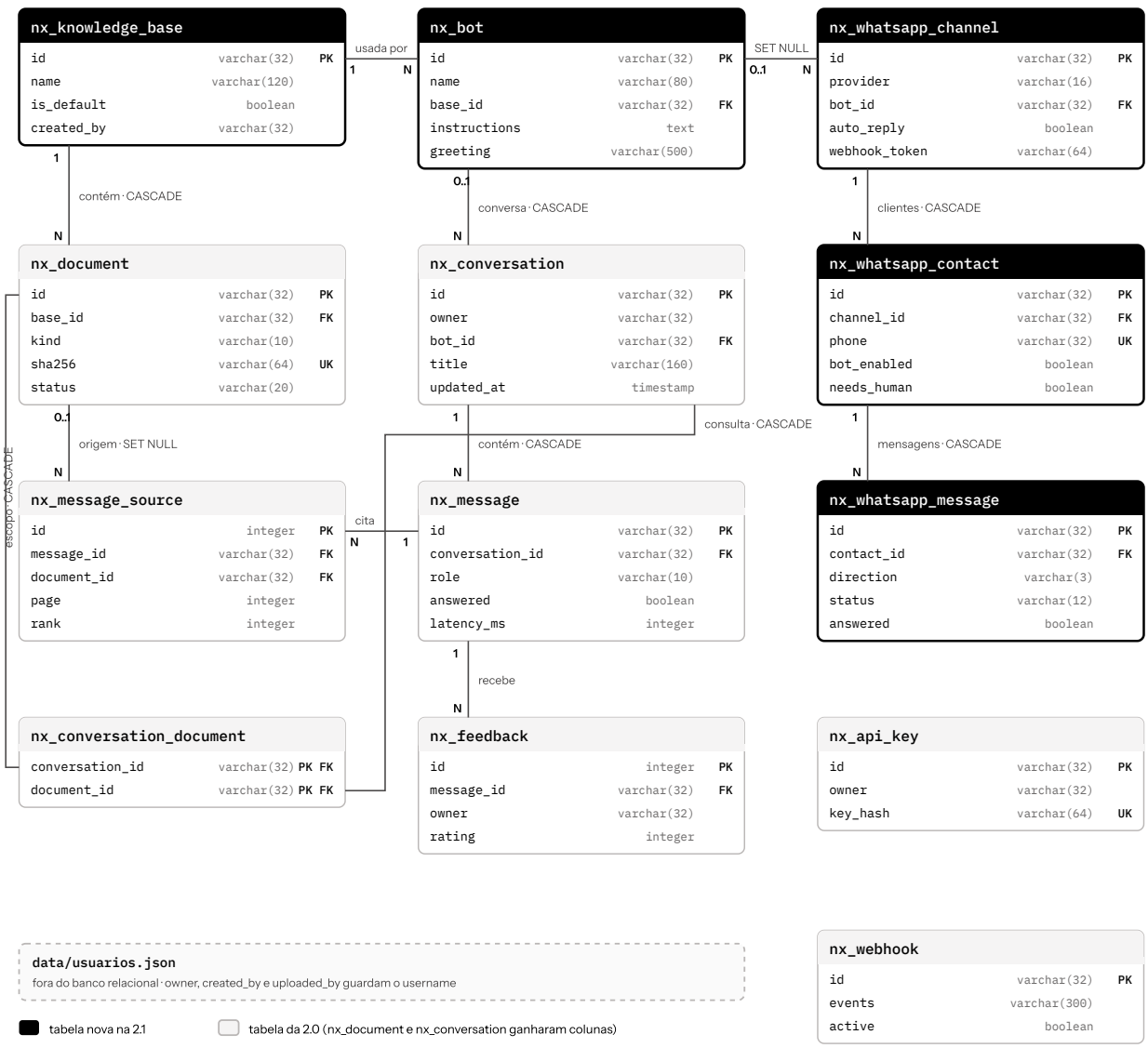


Figura 2.1 As treze tabelas do Nexo 2.1. As pretas são novas; "1", "0..1" e "N" indicam a cardinalidade. nx_api_key e nx_webhook se ligam às contas pelo nome de usuário.

Copyright (©) 2026 Erik Pimenta

03

Bases e acervo

Cada base de conhecimento reúne arquivos e textos. Cada item acompanha o arquivo em data/uploads (ou o texto na própria linha) e os trechos no banco vetorial.

3.1 nx_knowledge_base Nova

Uma linha por base. A base padrão (`is_default`) é criada sozinha, recebe os documentos antigos e os envios sem base, e não pode ser excluída.

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK
name	varchar(120)	não	—	Nome exibido; "Acervo geral" na base padrão
description	text	sim	—	Descrição curta
color	varchar(16)	não	#000000	Cor de destaque na interface
is_default	boolean	não	false	Verdadeiro só na base padrão
created_by	varchar(32)	não	—	Usuário que criou; ele e os administradores editam e excluem
created_at / updated_at	timestamp	não	agora (UTC)	Criação e última alteração

3.2 nx_document

Um registro por arquivo enviado ou texto criado numa base. As colunas marcadas como novas chegaram na 2.1.

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK. O arquivo fica em data/uploads/<id>.<extensão>
base_id **Nova**	varchar(32)	não	base padrão	FK para nx_knowledge_base, ON DELETE CASCADE (indexado)
kind **Nova**	varchar(10)	não	pdf	pdf, file (DOCX, TXT, MD, CSV, JSON, HTML) ou note (texto digitado)
extension **Nova**	varchar(10)	não	pdf	pdf, docx, txt, md, csv, json, html ou note
content **Nova**	text	sim	—	O texto, quando kind = note

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Padrão	Regra
title	varchar(200)	não	—	Título; entra no texto do embedding
filename	varchar(255)	não	—	Nome original, saneado (nos textos, o título com .txt)
sha256	varchar(64)	não	—	Único junto com base_id. Impede o mesmo conteúdo duas vezes na mesma base
size_bytes / pages / chunks	integer	não	— / 0 / 0	Tamanho, páginas (ou partes, fora do PDF) e trechos
status / error	varchar(20) / text	não / sim	PENDING	PENDING, PROCESSING, READY ou FAILED (indexado) e o motivo da falha
uploaded_by	varchar(32)	não	—	Usuário que enviou (indexado)
created_at / indexed_at	timestamp	não / sim	agora (UTC)	Envio e fim da indexação
updated_at **Nova**	timestamp	sim	—	Última edição de um texto

3.3 Ciclo do status

Ao iniciar, o servidor recoloca na fila os documentos que ficaram em PENDING ou PROCESSING (queda no meio da indexação), então nenhum envio se perde. Ao chegar em READY ou FAILED, a fila dispara o webhook correspondente (`document.ready` ou `document.failed`).

Renomear, reindexar e editar um texto voltam o item para PENDING (vale para READY e FAILED). Mover de base (troca de `base_id`) não muda o status, porque os vetores continuam valendo: a busca filtra pelos ids dos documentos.

Estado	Quando	O que vem depois
PENDING	Logo depois do envio, ao renomear, reindexar ou editar um texto, e ao reiniciar o servidor com itens pendentes	A fila pega o item
PROCESSING	A fila está lendo o arquivo, dividindo em trechos e gravando os vetores	READY ou FAILED
READY	Trechos gravados; o item entra nas respostas	Webhook document.ready
FAILED	Arquivo sem texto, protegido por senha ou com erro de leitura; o motivo fica em error	Webhook document.failed

Copyright (©) 2026 Erik Pimenta

04

Agentes

Um agente é um assistente com instruções próprias que responde só com uma base de conhecimento.

4.1 nx_bot Nova

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK
name	varchar(80)	não	—	Nome do agente
description	varchar(300)	sim	—	Descrição curta
base_id	varchar(32)	não	—	FK para nx_knowledge_base, sem cascata (indexado)
instructions	text	sim	—	Tom e regras somadas ao prompt estrito
greeting	varchar(500)	sim	—	Saudação; responde a cumprimentos
fallback	varchar(500)	sim	—	Frase usada quando a base não tem a resposta
color	varchar(16)	não	#000000	Cor do selo do agente
cite_sources	boolean	não	true	Acrescenta "Fontes:" nas respostas do WhatsApp
created_by	varchar(32)	não	—	Usuário que criou
created_at / updated_at	timestamp	não	agora (UTC)	Criação e última alteração

4.2 Como o agente usa a base

O agente não tem coleção vetorial própria. A cada pergunta, a aplicação lê os ids dos documentos prontos da base (`nx_document` com `base_id` do agente e `status = 'READY'`) e passa a lista como filtro `source_id IN (…)` para a busca vetorial. Se a lista estiver vazia, a pergunta não segue para a busca de todo o acervo: a API devolve 409 `BASE_EMPTY` .

Copyright (©) 2026 Erik Pimenta

05

Conversas e mensagens

Cada conversa pertence a um único usuário e, opcionalmente, a um agente, e guarda as perguntas, as respostas e os trechos que embasaram cada resposta.

5.1 nx_conversation

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK
owner	varchar(32)	não	—	Usuário dono (indexado)
bot_id **Nova**	varchar(32)	sim	—	FK para nx_bot, ON DELETE CASCADE (indexado). Nulo nas conversas gerais
title	varchar(160)	não	"Nova conversa"	Recebe a primeira pergunta
created_at / updated_at	timestamp	não	agora (UTC)	updated_at indexado para ordenar a lista

5.2 nx_conversation_document

Escopo das conversas gerais. Sem linhas, a conversa consulta todos os documentos prontos. Nas conversas com agente, o escopo é a base do agente e esta tabela não é usada.

Coluna	Tipo	Nulo	Regra
conversation_id	varchar(32)	não	PK e FK para nx_conversation, ON DELETE CASCADE
document_id	varchar(32)	não	PK e FK para nx_document, ON DELETE CASCADE

5.3 nx_message

Coluna	Tipo	Nulo	Regra
id	varchar(32)	não	PK
conversation_id	varchar(32)	não	FK para nx_conversation, ON DELETE CASCADE (indexado)

Copyright (©) 2026 Erik Pimenta

Coluna	Tipo	Nulo	Regra
role / content	varchar(10) / text	não	user ou assistant; texto com as citações [n]
answered / grounded	boolean	sim	false quando nada relevante foi encontrado; resultado da verificação de fidelidade
model / latency_ms	varchar(120) / integer	sim	Modelo que respondeu e tempo gasto
created_at	timestamp	não	Índice composto com conversation_id

5.4 nx_message_source

Coluna	Tipo	Nulo	Regra
id	integer	não	PK autoincremento
message_id	varchar(32)	não	FK para nx_message, ON DELETE CASCADE (indexado)
document_id	varchar(32)	sim	FK para nx_document, ON DELETE SET NULL (indexado)
document_title	varchar(200)	não	Cópia do título
chunk_id / page / rank	varchar(80) / integer	não	Id do trecho, página (ou parte) e número da citação
similarity / rerank_score	float	sim	Notas da busca e do reranker
excerpt	text	não	Texto do trecho (até 1.200 caracteres)

5.5 nx_feedback

Coluna	Tipo	Nulo	Regra
id	integer	não	PK autoincremento
message_id	varchar(32)	não	FK para nx_message, ON DELETE CASCADE
owner / rating	varchar(32) / integer	não	Único junto com message_id (uk_feedback_message_owner); 1 útil, -1 não ajudou
comment / created_at	text / timestamp	sim / não	Comentário opcional e data

Copyright (©) 2026 Erik Pimenta

06

WhatsApp

Três tabelas novas guardam as conexões de números, os clientes que escreveram e cada mensagem trocada, com a resposta do agente e as fontes.

6.1 nx_whatsapp_channel Nova

Uma linha por número conectado. Só administradores criam e alteram. Os segredos ficam aqui e nunca voltam inteiros pela API.

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK; faz parte da URL do webhook
name	varchar(80)	não	—	Nome da conexão
provider	varchar(16)	não	—	cloud, evolution ou simulador
bot_id	varchar(32)	sim	—	FK para nx_bot, ON DELETE SET NULL. Agente que responde
auto_reply / active	boolean	não	true / true	Resposta automática ligada; conexão ativa
phone	varchar(32)	sim	—	Número exibido
verify_token	varchar(64)	não	gerado	Token da confirmação do webhook da Meta
webhook_token	varchar(64)	não	gerado	Token na URL do webhook da Evolution
api_version / phone_number_id	varchar(16) / (64)	sim	v24.0 / —	API oficial: versão da Graph API e id do número
access_token / app_secret	text / varchar(200)	sim	—	API oficial: token de acesso e segredo do app (assinatura)
base_url / api_key / instance	varchar(300) / (200) / (80)	sim	—	Evolution: servidor, chave e instância
created_by / created_at	varchar(32) / timestamp	não	—	Quem criou e quando
last_event_at / last_error	timestamp / text	sim	—	Último evento recebido e último erro de envio

Copyright (©) 2026 Erik Pimenta

6.2 nx_whatsapp_contact Nova

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK
channel_id	varchar(32)	não	—	FK para nx_whatsapp_channel, ON DELETE CASCADE (indexado)
phone	varchar(32)	não	—	Número do cliente; único junto com channel_id (uk_whatsapp_contact_channel_phone)
name	varchar(120)	sim	—	Nome do perfil do WhatsApp ou o que a equipe definir
bot_enabled	boolean	não	true	Agente respondendo neste contato; falso quando a equipe assume
needs_human	boolean	não	false	Aguardando atendente
unread	integer	não	0	Mensagens do cliente ainda não abertas na tela
last_message_at	timestamp	sim	—	Ordena a caixa de entrada (indexado)
created_at	timestamp	não	agora (UTC)	Primeiro contato

6.3 nx_whatsapp_message Nova

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK
contact_id	varchar(32)	não	—	FK para nx_whatsapp_contact, ON DELETE CASCADE
direction	varchar(3)	não	—	in (cliente) ou out (agente ou equipe)
author	varchar(40)	não	—	cliente, agente ou o username de quem respondeu
body	text	não	—	Texto da mensagem
status	varchar(12)	não	—	received, pending, sent, delivered, read ou failed
external_id	varchar(128)	sim	—	Id da mensagem no provedor (indexado); atualiza a situação da entrega
error	text	sim	—	Erro devolvido pelo provedor
answered	boolean	sim	—	Nas respostas do agente: falso quando não encontrou
sources_json	text	sim	—	Fontes da resposta do agente, em JSON
created_at	timestamp	não	agora (UTC)	Índice composto com contact_id

Copyright (©) 2026 Erik Pimenta

6.4 Como uma mensagem é gravada

1. O webhook confere o token ou a assinatura, atualiza `last_event_at` e responde rápido.
2. O contato é criado ou atualizado (nome, `unread + 1`, `last_message_at`) e a mensagem entra como `in` / `received`.
3. Se a conexão tem agente, `auto_reply` e o contato `bot_enabled`, a resposta do agente entra como `out` / `pending` e passa a `sent` (com `external_id`) ou `failed` (com `error`).
4. Sem resposta na base, ou com pedido de atendente, o contato passa a `needs_human = true` e `bot_enabled = false`.
5. Atualizações de entrega da API oficial mudam `status` para `sent`, `delivered`, `read` ou `failed` pelo `external_id`.
6. Mensagens digitadas no próprio celular do número (Evolution, `fromMe`) entram como `out` com o autor "Pelo celular" e não acionam o agente.

Copyright (©) 2026 Erik Pimenta

07

Integrações

As chaves de API e os webhooks continuam como na 2.0. Os webhooks ganharam dois eventos do WhatsApp.

7.1 nx_api_key

Coluna	Tipo	Nulo	Padrão	Regra
id	varchar(32)	não	UUID4 hex	PK
owner	varchar(32)	não	—	Username do dono (indexado)
name / prefix	varchar(80) / (16)	não	—	Nome dado pelo usuário; 12 primeiros caracteres da chave
key_hash	varchar(64)	não	—	Único. SHA-256 da chave completa
created_at / last_used_at	timestamp	não / sim	agora (UTC)	Criação e último uso

Restrições e índices

- `PRIMARY KEY (id)`.
- `UNIQUE (key_hash)`: cada pedido localiza a chave por esse valor, em uma única leitura pelo índice da restrição.
- `ix_nx_api_key_owner` em `owner`: lista as chaves do usuário e a contagem de `integrations.api_keys` em `/api/system/status`.

7.2 Por que só o hash da chave é guardado

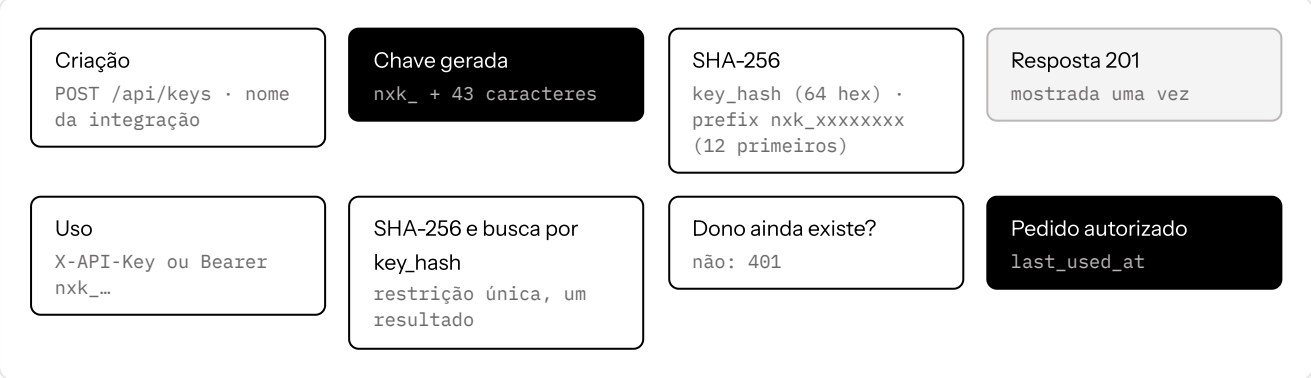


Figura 7.1 A chave completa só existe na resposta da criação. Daí em diante, o Nexo compara hashes.

Copyright (©) 2026 Erik Pimenta

- **Vazamento do banco não vira acesso.** Uma cópia do banco ou de um backup traz só hashes; com eles não se monta um pedido válido.
- **SHA-256 basta aqui.** Diferente de uma senha, a chave é aleatória e longa (43 caracteres depois de `nxk_`), então não há como adivinhá-la por dicionário. Um hash rápido e sem sal permite buscar a chave direto pela restrição única, sem percorrer a tabela. Senhas continuam em bcrypt.
- **Mostrada uma única vez.** Como o banco não tem a chave, não existe "ver de novo". Perdeu, revoga e cria outra.
- **O prefixo identifica sem expor.** Os 12 primeiros caracteres aparecem nas listas e ajudam a saber qual chave está num sistema, sem permitir o uso.
- **Conta excluída, chave apagada.** Excluir uma conta apaga as chaves dela. Mesmo assim, na autenticação, depois de achar o hash, o Nexo confere se o `owner` ainda existe em `usuarios.json`; se não existe, o pedido recebe 401.

7.3 nx_webhook

Coluna	Tipo	Nulo	Padrão	Regra
id / url	varchar(32) / (500)	não	UUID4 hex	PK; destino http ou https
events	varchar(300)	não	—	Subconjunto de document.ready, document.failed, question.unanswered, feedback.negative, whatsapp.message e whatsapp.handoff
secret / active	varchar(80) / boolean	não	gerado / true	Segredo do HMAC; webhook ligado
created_by / created_at	varchar(32) / timestamp	não	—	Quem cadastrou e quando
last_status / last_error / last_delivery_at	int / text / timestamp	sim	—	Resultado da última entrega

Sem índices além da chave primária: a tabela tem poucas linhas e é lida inteira a cada evento.

7.4 Como as entregas gravam last_status

Cada entrega roda numa thread do `ThreadPoolExecutor` (o teste roda na hora, dentro do pedido), usa `urllib.request` com tempo limite de `NEXO_WEBHOOK_TIMEOUT` segundos e, ao terminar, abre uma sessão própria no banco e atualiza as três colunas do webhook:

O que aconteceu	last_status	last_error	last_delivery_at
Receptor respondeu 2xx	o status (200, 204...)	nulo	agora (UTC)
Receptor respondeu 3xx, 4xx ou 5xx	o status (401, 500...)	descrição do erro HTTP	agora (UTC)

Copyright (©) 2026 Erik Pimenta

O que aconteceu	last_status	last_error	last_delivery_at
Tempo esgotado, DNS ou conexão recusada	nulo	a mensagem da falha	agora (UTC)

- As colunas guardam só a última entrega. Não há histórico de entregas nem nova tentativa automática.
- Entregas simultâneas para o mesmo webhook gravam na ordem em que terminam; vale a última.
- A falha de uma entrega nunca afeta a operação que a disparou: a pergunta, a avaliação, a indexação ou a mensagem do WhatsApp seguem normalmente.

Copyright (©) 2026 Erik Pimenta

08

Banco vetorial

Os trechos ficam numa coleção do ChromaDB (padrão) ou do Qdrant, com os embeddings e os metadados que ligam cada trecho ao item de origem.

8.1 Coleção

Campo	Conteúdo
id	UUID v5 de nexo:<id do documento>:<posição do trecho>; reindexar gera os mesmos ids
embedding	Vetor do modelo de NEXO_EMBED_MODEL, distância de cosseno
document	Texto do trecho
metadata.source_id	nx_document.id de origem; usado nos filtros de escopo, de base e na exclusão
metadata.title / filename / page / chunk_index	Título, nome do arquivo, página ou parte (a partir de 1) e posição do trecho

8.2 O que entra no embedding

Só o título do documento entra no texto usado para gerar o embedding, junto com o conteúdo do trecho. Isso ajuda perguntas como "no manual da bomba, ...". `source_id`, `filename`, `page` e `chunk_index` ficam fora do embedding e servem só para filtrar e citar. A base de conhecimento não entra nos metadados: ela é resolvida no banco relacional, pelo `base_id` de cada documento.

8.3 Filtros e exclusão

Operação	Como é feita
Conversa geral	Filtro source_id igual ao documento (um) ou dentro da lista (vários); sem escopo, consulta tudo
Agente e aba Testar	Filtro source_id dentro dos documentos prontos da base
Mover item de base	Nada muda na coleção: só a coluna base_id
Excluir item ou base	Apaga todos os vetores com source_id do item antes de apagar a linha e o arquivo
Reindexar ou editar texto	Apaga os vetores do item e grava os novos

Copyright (©) 2026 Erik Pimenta

8.4 Trocar o modelo de embeddings

Vetores de modelos diferentes não são comparáveis. Depois de mudar `NEXO_EMBED_PROVIDER` ou `NEXO_EMBED_MODEL`, use uma coleção nova (`NEXO_COLLECTION`) ou reindexe todos os documentos pela tela Documentos. Bases, agentes e conversas não precisam de nenhum ajuste, porque guardam só os ids dos documentos.

Copyright (©) 2026 Erik Pimenta

09

Contas e arquivos

As contas ficam num arquivo JSON, fora do banco relacional, para que o login funcione antes de qualquer banco externo.

9.1 data/usuarios.json

```
{
  "versao": 1,
  "usuarios": [
    {
      "username": "admin",
      "name": "Administrador",
      "role": "admin",
      "password_hash": "$2b$12$...",
      "created_at": "2026-01-01T12:00:00+00:00",
      "last_login_at": null,
      "tutorial_done_at": "2026-10-30T14:02:11+00:00"
    }
  ]
}
```

Campo	Regra
username	3 a 32 caracteres minúsculos; referenciado por owner, uploaded_by e created_by
role	admin ou user; o último admin não pode ser rebaixado
password_hash	Hash bcrypt
name	Nome exibido na interface
created_at / last_login_at	ISO 8601 em UTC
tutorial_done_at **Novo**	Quando a conta concluiu ou pulou o tutorial; nulo mostra o tutorial na próxima entrada. Arquivos antigos, sem o campo, valem como nulo

- O arquivo é regravado de forma atômica: a versão nova vai para um arquivo temporário na mesma pasta e substitui o antigo de uma vez. Uma trava em memória impede duas gravações ao mesmo tempo dentro do processo.
- Se não houver nenhum administrador (por exemplo, na primeira execução), o Nexo cria o login pré-definido `admin` / `Nexo@2026`.
- Excluir uma conta (`DELETE /api/users/{username}`) remove a entrada do arquivo e apaga do banco as chaves de API e as conversas dela (com mensagens, fontes e avaliações, em cascata).
- Para redefinir uma senha esquecida: `python scripts/redefinir_senha.py nome-do-usuario`.

Copyright (©) 2026 Erik Pimenta

9.2 data/uploads

Cada arquivo é gravado como `<id>.<extensão>` (por exemplo `<id>.pdf` ou `<id>.docx`), com o id de `nx_document`. O nome original fica só na coluna `filename`, o que evita conflitos e nomes perigosos no disco. Os textos criados na aba Textos não geram arquivo: ficam só na coluna `content`.

9.3 data/.segredo

Chave aleatória criada quando `NEXO_SECRET` está vazio. Apagar o arquivo invalida todas as sessões abertas. Chaves de API não dependem dele: continuam válidas porque são conferidas pelo hash. Os tokens das conexões do WhatsApp também não dependem dele.

Copyright (©) 2026 Erik Pimenta

10

Consultas e índices

Cada índice existe por causa de uma consulta da aplicação. As versões em SQL abaixo servem para relatórios e conferência.

10.1 Consultas principais

Tela ou rotina	Consulta	Índice usado
Lista de conversas	nx_conversation por owner (e bot_id nos chats com agentes), ordenada por updated_at desc	ix_nx_conversation_owner, ix_nx_conversation_updated_at, ix_nx_conversation_bot_id
Abrir ou exportar conversa	nx_message por conversation_id, ordenada por created_at	ix_nx_message_conversation_created
Fontes de uma resposta	nx_message_source por message_id, ordenada por rank	ix_nx_message_source_message_id
Excluir documento	nx_message_source por document_id (SET NULL)	ix_nx_message_source_document_id
Envio de arquivo	nx_document por base_id e sha256	restrição única uk_document_base_sha256
Itens de uma base e escopo do agente	nx_document por base_id (e status READY)	ix_nx_document_base_id, ix_nx_document_status
Agentes de uma base	nx_bot por base_id (contagem e bloqueio da exclusão)	ix_nx_bot_base_id
Retomar a fila	nx_document com status em PENDING ou PROCESSING	ix_nx_document_status
Sistema e conta	contagem de nx_document por status e soma de chunks; chaves do usuário; total de webhooks	ix_nx_document_status, ix_nx_api_key_owner
Autenticar chave de API	nx_api_key por key_hash	restrição única de key_hash
Disparo de evento	nx_webhook com active verdadeiro, filtrada pelo evento em Python	leitura completa (tabela pequena)
Caixa de entrada do WhatsApp	nx_whatsapp_contact por channel_id, ordenada por last_message_at desc	ix_nx_whatsapp_contact_channel_id, ix_nx_whatsapp_contact_last_message_at
Mensagem recebida	nx_whatsapp_contact por channel_id e phone	restrição única uk_whatsapp_contact_channel_phone

Copyright (©) 2026 Erik Pimenta

Tela ou rotina	Consulta	Índice usado
Conversa com o cliente	nx_whatsapp_message por contact_id, ordenada por created_at	ix_nx_whatsapp_message_contact_created
Situação da entrega	nx_whatsapp_message por external_id	ix_nx_whatsapp_message_external_id
Painel de qualidade	nx_message do período, unida a nx_conversation para o escopo	ix_nx_conversation_owner, ix_nx_message_conversation_created

10.2 Lista de índices

Índice	Tabela	Colunas
ix_nx_document_status	nx_document	status
ix_nx_document_uploaded_by	nx_document	uploaded_by
ix_nx_document_base_id **Novo**	nx_document	base_id
uk_document_base_sha256 **Novo**	nx_document	base_id, sha256 (único; substitui o único de sha256)
ix_nx_bot_base_id **Novo**	nx_bot	base_id
ix_nx_conversation_owner / _updated_at	nx_conversation	owner; updated_at
ix_nx_conversation_bot_id **Novo**	nx_conversation	bot_id
ix_nx_message_conversation_created	nx_message	conversation_id, created_at
ix_nx_message_source_message_id / _document_id	nx_message_source	message_id; document_id
uk_feedback_message_owner	nx_feedback	message_id, owner (único)
ix_nx_api_key_owner e único de key_hash	nx_api_key	owner; key_hash
ix_nx_whatsapp_contact_channel_id **Novo**	nx_whatsapp_contact	channel_id
ix_nx_whatsapp_contact_last_message_at **Novo**	nx_whatsapp_contact	last_message_at
uk_whatsapp_contact_channel_phone **Novo**	nx_whatsapp_contact	channel_id, phone (único)
ix_nx_whatsapp_message_contact_created **Novo**	nx_whatsapp_message	contact_id, created_at
ix_nx_whatsapp_message_external_id **Novo**	nx_whatsapp_message	external_id

10.3 Consultas úteis da 2.1 Novo

Conteúdo por base

Copyright (©) 2026 Erik Pimenta

```
SELECT b.name AS base,
       SUM(CASE WHEN d.kind = 'pdf' THEN 1 ELSE 0 END) AS pdfs,
       SUM(CASE WHEN d.kind = 'file' THEN 1 ELSE 0 END) AS arquivos,
       SUM(CASE WHEN d.kind = 'note' THEN 1 ELSE 0 END) AS textos,
       SUM(CASE WHEN d.status = 'READY' THEN d.chunks ELSE 0 END) AS trechos
FROM nx_knowledge_base b
LEFT JOIN nx_document d ON d.base_id = b.id
GROUP BY b.name
ORDER BY b.name;
```

Agentes por base

```
SELECT b.name AS base, COUNT(a.id) AS agentes
FROM nx_knowledge_base b
LEFT JOIN nx_bot a ON a.base_id = b.id
GROUP BY b.name;
```

Contatos aguardando atendente

```
SELECT c.name, c.phone, ch.name AS conexao, c.last_message_at
FROM nx_whatsapp_contact c
JOIN nx_whatsapp_channel ch ON ch.id = c.channel_id
WHERE c.needs_human = TRUE
ORDER BY c.last_message_at DESC;
```

Respostas do agente por dia no WhatsApp

```
SELECT DATE(m.created_at) AS dia,
       COUNT(*) AS respostas,
       SUM(CASE WHEN m.answered = FALSE THEN 1 ELSE 0 END) AS sem_resposta
FROM nx_whatsapp_message m
WHERE m.direction = 'out' AND m.author = 'agente' AND m.created_at >= :inicio
GROUP BY DATE(m.created_at)
ORDER BY dia;
```

10.4 Consultas do painel de qualidade

Perguntas, respondidas e taxa de resposta

```
SELECT COUNT(*) AS respostas,
       SUM(CASE WHEN m.answered THEN 1 ELSE 0 END) AS respondidas,
       ROUND(AVG(CASE WHEN m.answered THEN 1.0 ELSE 0 END), 3) AS taxa
FROM nx_message m
JOIN nx_conversation c ON c.id = m.conversation_id
WHERE m.role = 'assistant' AND m.created_at >= :inicio;
```

Documentos mais citados

Copyright (©) 2026 Erik Pimenta

```
SELECT s.document_id, MAX(s.document_title) AS titulo, COUNT(*) AS citacoes
FROM nx_message_source s
JOIN nx_message m ON m.id = s.message_id
WHERE m.created_at >= :inicio
GROUP BY s.document_id
ORDER BY citacoes DESC
LIMIT 6;
```

Perguntas e perguntas sem resposta por dia

```
SELECT DATE(m.created_at) AS dia,
       COUNT(*) AS perguntas,
       SUM(CASE WHEN m.answered = FALSE THEN 1 ELSE 0 END) AS sem_resposta
FROM nx_message m
JOIN nx_conversation c ON c.id = m.conversation_id
WHERE m.role = 'assistant' AND m.created_at >= :inicio
GROUP BY DATE(m.created_at)
ORDER BY dia;
```

O painel conta perguntas pelas mensagens `user` e completa com zero os dias sem atividade; contar pelas respostas, como acima, dá o mesmo número quando toda pergunta foi respondida ou recusada.

Lacunas do acervo: perguntas sem resposta mais recentes

```
SELECT q.content AS pergunta, q.created_at AS feita_em, q.conversation_id
FROM nx_message a
JOIN nx_message q ON q.conversation_id = a.conversation_id
 AND q.role = 'user'
 AND q.created_at = (SELECT MAX(x.created_at) FROM nx_message x
                     WHERE x.conversation_id = a.conversation_id
                       AND x.role = 'user' AND x.created_at <= a.created_at)
WHERE a.role = 'assistant' AND a.answered = FALSE AND a.created_at >= :inicio
ORDER BY q.created_at DESC
LIMIT 10;
```

Respostas avaliadas como "não ajudou"

```
SELECT f.created_at, f.owner, m.id AS message_id, m.conversation_id, m.content AS resposta
FROM nx_feedback f
JOIN nx_message m ON m.id = f.message_id
WHERE f.rating = -1 AND f.created_at >= :inicio
ORDER BY f.created_at DESC
LIMIT 10;
```

Latência média

```
SELECT ROUND(AVG(latency_ms)) AS media_ms
FROM nx_message
WHERE role = 'assistant' AND latency_ms IS NOT NULL AND created_at >= :inicio;
```

Por que o p95 fica em Python. Percentis têm sintaxe diferente em cada banco (`percentile_cont` no PostgreSQL, nenhuma função no SQLite). O Nexo lê as latências do período e calcula o p95 na aplicação, o que funciona igual nos quatro bancos suportados. Documentos excluídos aparecem com `document_id` nulo e o título

Copyright (©) 2026 Erik Pimenta

copiado. Para o escopo de um usuário, acrescente `AND c.owner = 'nome'`; para um agente, `AND c.bot_id = 'id'`.

Copyright (©) 2026 Erik Pimenta

11

Decisões de arquitetura

Por que o banco é como é. Cada decisão tem um custo, e ele está escrito ao lado.

11.1 Relacional e vetorial separados

O banco relacional guarda o que precisa de integridade e histórico (bases, documentos, agentes, conversas, citações, integrações e WhatsApp); o vetorial guarda só o que serve à busca. A ligação entre os dois é o `source_id` nos metadados e o `chunk_id` nas fontes. Assim, dá para trocar ChromaDB por Qdrant sem mexer nas tabelas.

11.2 Fontes gravadas com cópia do texto

`nx_message_source` guarda o título e o trecho, não só a referência. Uma resposta antiga continua auditável, e exportável, mesmo depois que o documento é excluído ou reindexado com outro tamanho de trecho. As respostas do agente no WhatsApp seguem a mesma ideia, com as fontes copiadas em `sources_json`.

11.3 Contas em arquivo

Sem tabela de usuários, o primeiro acesso funciona antes de qualquer banco externo e a redefinição de senha é um script de uma linha. O preço é que `owner`, `uploaded_by` e `created_by` não têm chave estrangeira: a aplicação confere o usuário a cada pedido, inclusive nos pedidos com chave de API.

11.4 IDs gerados pela aplicação

UUIDs em hexadecimal permitem gravar o arquivo em disco antes do commit e evitam colisões ao juntar bancos. As tabelas de alto volume e sem exposição externa (`nx_message_source`, `nx_feedback`) usam autoincremento, que ocupa menos espaço. O id da conexão do WhatsApp entra na URL do webhook, por isso também é aleatório.

11.5 Escopo por ausência

Uma conversa geral sem linhas em `nx_conversation_document` consulta tudo o que está pronto. Documentos novos entram automaticamente nas conversas abertas, sem atualizar escopos.

11.6 Chaves de API como hash

Guardar só o SHA-256 torna inofensivo um vazamento do banco ou de um backup. O custo é não poder mostrar a chave de novo; o prefixo de 12 caracteres compensa na hora de identificar qual chave está em qual sistema.

Copyright (©) 2026 Erik Pimenta

11.7 Webhooks sem fila persistente

As entregas saem de um pool de threads em memória e o resultado fica na própria linha do webhook. É simples e não exige outro serviço (Redis, fila de mensagens). O custo: um evento em trânsito se perde se o servidor parar naquele instante, e não há histórico de entregas. Para quem precisa de garantia, o receptor pode conferir o estado pela API (por exemplo, listar documentos ou contatos aguardando atendente) ao reiniciar.

11.8 Métricas calculadas na hora

O painel não tem tabela de agregados: cada consulta lê as mensagens do período e calcula em Python. Os números estão sempre certos e não há rotina para manter. Com muitos milhões de mensagens, períodos longos ficam mais lentos; aí vale passar para PostgreSQL e limitar o período.

11.9 Filtro do agente pelos ids dos documentos da base Novo

Em vez de uma coleção vetorial por base, todos os trechos ficam na mesma coleção e o agente filtra pelos ids dos documentos prontos da base dele. Mover um item de base é só trocar `base_id`, sem reindexar, e uma base nova não exige criar nada no banco vetorial. O custo é uma lista `IN` maior em bases muito grandes, que os dois bancos vetoriais resolvem bem.

11.10 Segredos do WhatsApp mascarados na API Novo

Tokens, app secret e chaves da Evolution ficam no banco porque o Nexo precisa deles para enviar mensagens e conferir assinaturas, mas nunca voltam inteiros pela API: só os quatro últimos caracteres. Editar com o campo vazio mantém o valor. O custo é que o backup do banco contém esses segredos e precisa ser guardado como uma senha.

11.11 Único de sha256 por base Novo

Na 2.0, o mesmo arquivo não podia ser enviado duas vezes em todo o acervo. Na 2.1, a restrição vale por base (`uk_document_base_sha256`): o mesmo manual pode estar na base do Suporte e na do Comercial, cada cópia com os próprios vetores, e continua impossível repeti-lo dentro da mesma base.

Copyright (©) 2026 Erik Pimenta

12

Migração automática

Bancos criados pela 2.0 sobem na 2.1 sem nenhum passo manual. A migração roda a cada inicialização e só faz o que falta.

12.1 O que a migração faz

1. Cria as tabelas novas com `create_all`.
2. Cria a base "Acervo geral" (`is_default = true`) se não existir nenhuma base padrão.
3. Em `nx_document`, acrescenta `base_id`, `kind`, `extension`, `content` e `updated_at`, preenche `base_id` com a base padrão, `kind = 'pdf'` e `extension = 'pdf'`, e troca o único de `sha256` por `uk_document_base_sha256`.
4. Em `nx_conversation`, acrescenta `bot_id`.
5. Cria os índices novos que faltarem.

12.2 SQLite: os 12 passos oficiais

O SQLite não remove uma restrição única com `ALTER TABLE`. Por isso a migração reconstrói `nx_document` seguindo o procedimento oficial do SQLite: desliga as chaves estrangeiras, abre uma transação, cria `nx_document_nova` com a estrutura da 2.1, copia os dados preenchendo as colunas novas, apaga a tabela antiga, renomeia a nova para `nx_document`, recria os índices, confere `PRAGMA foreign_key_check`, confirma a transação e religa as chaves estrangeiras. As tabelas filhas continuam apontando para `nx_document`.

12.3 Outros bancos

No PostgreSQL, MySQL e SQL Server, a migração usa `ALTER TABLE … ADD COLUMN`, descobre o nome da restrição única antiga pelo inspetor do SQLAlchemy, remove essa restrição e cria a nova. Faça um backup antes de atualizar, como em qualquer migração.

Copyright (©) 2026 Erik Pimenta

13

Fluxos de dados

O que cada operação grava, na ordem em que grava.

13.1 Envio de um arquivo para uma base

1. A API confere a extensão, o tamanho, o cabeçalho %PDF dos PDFs e o SHA-256 dentro da base (duplicado devolve 409 DUPLICATE).
2. Grava data/uploads/<id>.<extensão> e a linha em nx_document com PENDING.
3. A fila extrai o texto, divide em trechos e grava os vetores; termina em READY ou FAILED e dispara o webhook.

13.2 Uma pergunta a um agente

1. Grava a pergunta em nx_message (role = user) na conversa do agente.
2. Lê os documentos prontos da base do agente, busca com esse filtro, aplica os cortes e o reranking.
3. Grava a resposta e uma linha em nx_message_source por trecho citado; sem resposta, dispara question.unanswered.

13.3 Pergunta direta e busca

Chamada	O que é gravado
POST /api/ask sem conversation_id	Nenhuma mensagem. Só last_used_at da chave, se o pedido veio com chave; sem resposta, dispara question.unanswered com conversation_id nulo
POST /api/ask com conversation_id	O mesmo que uma pergunta no chat (seção 13.2)
POST /api/ask com bot_id	O mesmo, usando a base e as instruções do agente
POST /api/search	Nada. A busca só lê o banco vetorial (e nx_document, quando há base_id)
GET /api/conversations/{id}/export	Nada. Lê conversa, mensagens e fontes
POST /api/whatsapp/contacts/{id}/suggest	Nada. A sugestão do agente não é gravada nem enviada

13.4 Avaliação

Grava ou atualiza a linha de `nx_feedback` do usuário para a resposta (rating 0 apaga). Quando a avaliação passa a ser −1, dispara `feedback.negative` com a pergunta e a resposta; repetir −1 não dispara de novo.

Copyright (©) 2026 Erik Pimenta

13.5 Exclusões

O que é excluído	Efeito
Base	409 se algum agente usa; senão, vetores e arquivos dos itens apagados e itens apagados (CASCADE)
Item da base	Vetores apagados, arquivo removido, linha apagada, escopos apagados, fontes com document_id nulo
Agente	Conversas do agente apagadas (CASCADE); conexões do WhatsApp ficam sem agente (SET NULL)
Conexão do WhatsApp	Contatos e mensagens apagados (CASCADE)
Conversa	Escopo, mensagens, fontes e avaliações apagados (CASCADE)
Avaliação (rating 0)	Só a linha de nx_feedback
Chave de API	Só a linha de nx_api_key; pedidos com ela recebem 401 na hora
Webhook	Só a linha de nx_webhook; entregas já em andamento terminam, mas o resultado não tem mais onde ser gravado
Conta	Entrada de usuarios.json, linhas de nx_api_key do dono e as conversas dele, com mensagens, fontes e avaliações (CASCADE)

Copyright (©) 2026 Erik Pimenta

14

Outros bancos e escala

Como trocar o SQLite por um servidor de banco, migrar os dados e acompanhar o crescimento.

14.1 Outro banco relacional

Banco	Driver	Exemplo de URL
PostgreSQL	pip install "psycopg[binary]"	postgresql+psycopg://nexo:senha@localhost:5432/nexo
MySQL / MariaDB	pip install pymysql	mysql+pymysql://nexo:senha@localhost:3306/nexo
SQL Server	pip install pyodbc	mssql+pyodbc://nexo:senha@servidor/nexo?driver=ODBC+Driver+18+for+SQL+Server

14.2 Migrando dados do SQLite

1. Pare o servidor e faça uma cópia de data/nexo.db.
2. Crie o banco novo com docs/schema-postgresql.sql (ou deixe o Nexo criar ao iniciar com a URL nova).
3. Copie as tabelas na ordem das chaves: nx_knowledge_base, nx_document, nx_bot, nx_conversation, nx_conversation_document, nx_message, nx_message_source, nx_feedback, nx_api_key, nx_webhook, nx_whatsapp_channel, nx_whatsapp_contact e nx_whatsapp_message.
4. No PostgreSQL, ajuste as sequências de nx_message_source.id e nx_feedback.id para o maior id copiado.
5. Defina NEXO_DATABASE_URL e suba o servidor. O banco vetorial e os arquivos não mudam.

14.3 Crescimento

Tabela ou coleção	Cresce com	O que fazer quando ficar grande
Coleção vetorial	Páginas e partes indexadas	Qdrant em servidor (NEXO_QDRANT_URL)
nx_message e nx_message_source	Perguntas feitas	PostgreSQL; apagar conversas antigas apaga as fontes em cascata
nx_whatsapp_message	Mensagens de clientes	PostgreSQL; arquivar contatos antigos apagando a conexão de testes ou os contatos
nx_document	Arquivos e textos	Cresce pouco; o peso está em data/uploads

Copyright (©) 2026 Erik Pimenta

14.4 Backup

Pare o servidor e copie a pasta `data` inteira. Com bancos externos, faça também o backup deles no mesmo momento. O backup contém os segredos dos webhooks e os tokens do WhatsApp: guarde-o com o mesmo cuidado de uma senha.

14.5 DDL de referência

Trecho de docs/schema-postgresql.sql

```
CREATE TABLE nx_bot (
    id VARCHAR(32) NOT NULL,
    name VARCHAR(80) NOT NULL,
    description VARCHAR(300),
    base_id VARCHAR(32) NOT NULL,
    instructions TEXT,
    greeting VARCHAR(500),
    fallback VARCHAR(500),
    color VARCHAR(16) NOT NULL,
    cite_sources BOOLEAN NOT NULL,
    created_by VARCHAR(32) NOT NULL,
    created_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    updated_at TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY(base_id) REFERENCES nx_knowledge_base (id)
);
```

Copyright (©) 2026 Erik Pimenta